Classic planning with PDDL

Aleksandra Słomska
Zofia Narloch

March 4, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
```

Listing 1: Outcome of the first problem - one person and one box

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( PICK-UP-CRATE CRATE2 DEPOT DRONE1 LEFT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
( DROP-CRATE ROOM1 PERSON2 CRATE2 LEFT MEDICINE DRONE1 )
```

Listing 2: Outcome of the second problem - two people and three boxes

2 Exercise 1.2: Problem Builder in Python

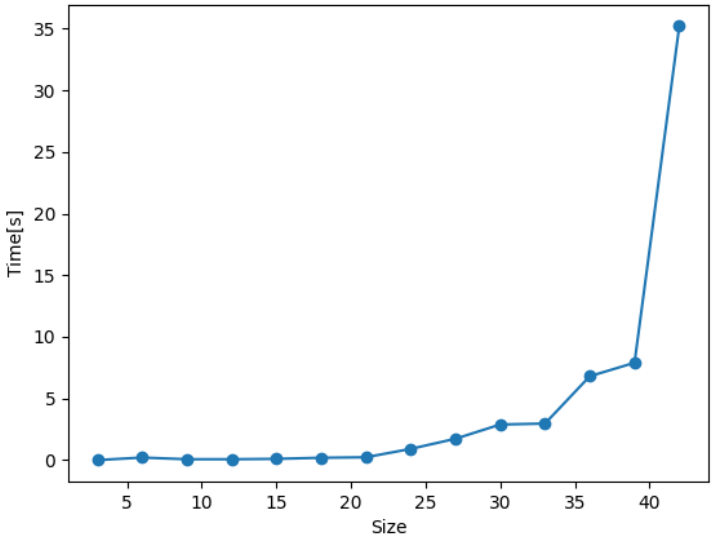

Figure 1: Size of the problem and the time required to find a solution in the tests

3 Exercise 1.3: Performance Comparison of Search Algorithms and Heuristics

3.1 First Part

Algorithm	Heuristic	Largest Solved Size	Time (s)	Plan Length	Optimal
BFS	None	7	21.07	23	Yes
IDS	None	3	0.43	10	Yes
A*	hMAX	5	6.51	16	Yes
GBFS	hMAX	7	8.66	26	No

Table 1: Performance Comparison of Search Algorithms and Heuristics (Exercise 1.3.1)

The results of the performance of every algorithm show that only GBFS with heuristic hMAX is not optimal. The reason for that is the longest plan length. This is because GBFS relies on the heuristic to rush toward the goal quickly rather than exhaustively searching for the perfect path. The same size of the

problem was solved by the BFS algorithm without any heuristics. However, it took more time, because it blindly checks every possible combination of moves. On the other hand, the A* algorithm only reached size 5 because the calculation of the hMAX heuristic for every node caused it to time out. Algorithm IDS solved the smallest problem of them all. IDS intentionally wipes its memory and starts over every time it increases its search depth, so it concentrates on memory efficiency.

In this part every action in domain file has the same cost, that is why algorithms like BFS guarantee the shortest path by expanding all nodes at the current search depth before moving deeper. Secondly, at every step, the drone has many valid moves, which creates many possibilities, which can cause problems for some algorithms like BFS.

3.2 Second Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
GBFS	NONE	0.27	24
GBFS	hMAX	Timeout	$>$ 60s
GBFS	hFF	0.30	24
GBFS	hADD	0.29	27
GBFS	LANDMARK	0.20	28
EHC	NONE	0.20	27
EHC	hMAX	Timeout	$>$ 60s
EHC	hFF	0.21	27
EHC	hADD	0.19	24
EHC	LANDMARK	0.44	35

Table 2: Performance Comparison of GBFS and EHC with Various Heuristics (Exercise 1.3.2)

Based on the results from Exercise 1.3.1, the largest problem that Greedy Best-First Search (GBFS) could handle within the 1-minute time limit was size 7. This problem size was generated and tested for the algorithms GBFS and EHC with heuristics hMAX, hFF, hADD and Landmark. Both algorithms with heuristic hMAX exceeded the time limit, despite GBFS solving a size 7 problem with hMAX in under 9 seconds during the previous exercise. This occurred because the problem generator places elements randomly.

EHC is more memory-efficient than GBFS, since it discards alternative paths once it makes a choice. It causes the algorithm to be fast and use minimal memory. However, it causes problems when the chosen path reaches dead-end, since it can't go back. The GBFS keeps all values in memory, so it can return if the path is unpromising, but at cost of taking much more memory.

The FF planer intelligently combines the strengths of both algorithms to maximize speed and reliability. The FF planner primarily relies on EHC paired with the hFF heuristic. However, if EHC gets trapped in a dead-end, the planner

automatically changes to GBFS. The result of that is the planner can backtrack out of the dead-end, guaranteeing that a solution will eventually be found.

3.3 Third Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
A*	NONE	2.57	16
BFS	NONE	0.82	16
IDS	NONE	Timeout	> 60s
A*	hMAX	6.60	16
A*	LMCUT	17.91	16

Table 3: Performance of Optimal Planners and Admissible Heuristics (Exercise 1.3.3)

An admissible heuristic is one that never overestimates the true cost to reach the goal, guaranteeing that an algorithm like A* will find an optimal solution. Among the heuristics implemented in Pyperplan, hMAX and LMCUT (Landmark-Cut) are admissible.

The results show that almost every algorithm has the plan length equal to 16. The best-performing algorithm was BFS without any heuristics. The reason for that is the same cost for every action. Moreover, adding the heuristics to algorithm A* resulted in exceeding the time 3-6 times more. It is because of the necesity to calculate these complex heuristics at every node.

Finally, IDS timed out because a plan length of 16 steps requires it to wipe its memory and completely regenerate the search tree from scratch 16 times, which causes to exceed the 1-minute limit.

4 Exercise 2.1: Problem Builder in Python

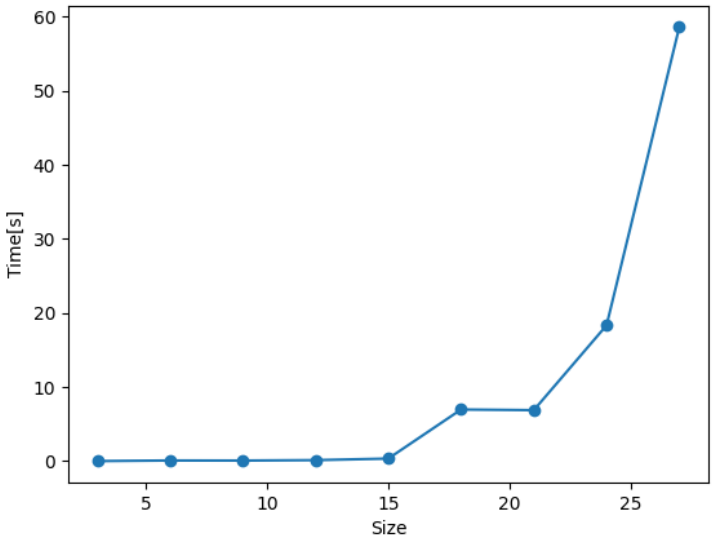

Figure 2: Size of the problem and the time required to find a solution in the tests

5 Exercise 2.2: Problem Builder in Python

Test run with lama-first alias

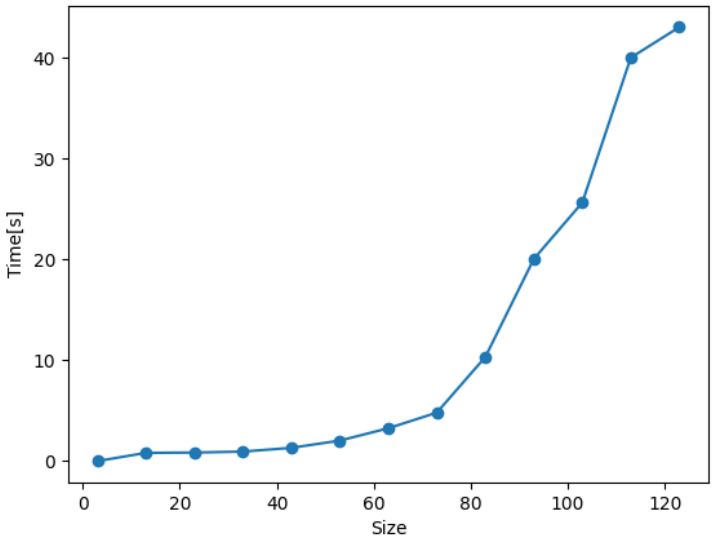

Figure 3: Size of the problem and the time required to find a solution in the tests